PG Management SaaS: Build Plan

A plan for building operator software for PG hostel owners in Mumbai and Hyderabad. Owners pay a monthly subscription, and it solves a daily pain: rent chasing, vacant beds, and tenant records kept in notebooks and WhatsApp.

1. Who uses it

- **Owner/operator:** runs one or many PGs and wants money and occupancy at a glance
- **Manager/warden:** day-to-day tasks, complaints, check-ins
- **Tenant:** pays rent, raises complaints, sees notices (a light app or WhatsApp-only at first)

2. MVP modules (in priority order)

1. **Property, room and bed setup:** floors, rooms, beds, sharing type, rent, deposit, and a visual occupancy grid
2. **Tenant management:** onboarding, KYC documents, join and exit dates, emergency contacts, agreement upload
3. **Rent and billing:** monthly rent cycles, due dates, partial payments, deposit tracking, electricity or food add-ons, receipts, and an outstanding-dues dashboard
4. **Payment collection:** UPI payment links and QR codes (Razorpay or Cashfree), auto-reconciliation, plus cash/manual entry
5. **Reminders:** automatic WhatsApp and SMS reminders for dues. This is the feature owners value most.
6. **Complaints and notices:** tenants raise tickets, owners broadcast announcements
7. **Reports:** occupancy, collection vs pending, expenses, monthly P&L

Later: expense tracking, staff and mess management, meal plans, visitor logs, vacate/notice workflow, a public listing page per PG (where the marketplace idea can slot in), and multi-property analytics.

3. Tech stack (lean)

- **Owner app:** a mobile-first web app (PWA) with Next.js, since owners mostly use phones but you avoid app store friction early. Add Flutter or React Native once the workflows settle.
- **Backend:** NestJS or FastAPI, with PostgreSQL
- **Auth:** phone OTP, with role-based access (owner / manager / tenant)
- **Multi-tenancy:** a single database with an `organization_id` on every table, and row-level checks. This is the key architectural decision for SaaS.
- **Jobs:** a queue (BullMQ or Celery) for monthly rent generation and reminders
- **Messaging:** WhatsApp Business API through a provider such as Gupshup, Interakt or WATI
- **Payments:** Razorpay or Cashfree payment links with webhooks
- **Files:** S3 for KYC documents, encrypted
- **Fast start:** Next.js + Supabase works well, since row-level security handles multi-tenancy neatly

4. Core data model

- `organizations` (the PG business) and `users` with roles
- `properties` → `floors` → `rooms` → `beds`
- `tenants` and `stays` (tenant ↔ bed, start/end dates, rent, deposit). Model stays separately so bed history is kept.
- `rent_invoices` (per stay per month: amount, due date, status)
- `payments` (invoice, amount, mode, gateway reference)
- `charges` (electricity, food, late fees, one-offs)
- `complaints`, `notices`, `expenses`
- `subscriptions` (your own SaaS billing to owners)

5. Business model

- **Pricing:** per-bed per-month (for example ₹10–20 per bed) or tiered plans by bed count. Per-bed scales naturally with the owner's size.
- **Free tier:** up to about 10–15 beds to get small owners hooked

- **Extra revenue:** a small fee on rent collected through the platform, and add-ons like WhatsApp automation or agreement generation

6. India-specific points

- Owners trust WhatsApp and UPI. Build around them, not around email and cards.
- Many owners use Excel or notebooks, so offer **Excel/CSV import** for tenants and rents to make onboarding painless.
- Support Hindi, Marathi and Telugu labels in the owner app eventually.
- Store Aadhaar and ID documents carefully: encrypted, limited access, and a clear consent flow (DPDP Act).
- Rent practices vary: advance vs arrears billing, pro-rated first month, and notice periods. Make these configurable per property.

7. Build plan

Timeline	Focus
Weeks 0–2	Interview 10–15 PG owners in Mumbai and Hyderabad about how they track rent today; sketch the screens
Weeks 3–8	Build modules 1–3 plus manual payment entry and a dashboard
Weeks 9–10	Add UPI collection and WhatsApp reminders
Weeks 11–12	Pilot with 5–10 owners, free, in exchange for feedback
Month 4+	Start charging, then add complaints, expenses and reports

8. Biggest risks

- **Adoption:** owners must enter data, so onboarding has to take under 30 minutes (bulk import, templates)

- **Tenant payment behaviour:** if tenants don't pay through the app, owners drift back to cash. Make the payment link one tap from the WhatsApp reminder.
- **Trust:** you hold financial and ID data, so security basics (encryption, audit logs, backups) can't wait

9. Next steps

- Mock up the key screens: occupancy grid, tenant list, rent dues dashboard
- Write out the full database schema